AI-ASSISTED CODING

ASSIGNMENT-16.3

Database Design and Queries: Schema Design and SQL
Generation

NAME: PALITHA VIKAS

HALL-TICKET: 2403A510E6

BATCH NO: 05

Task 1 – Student Information System Schema

Design a database schema for a Student Information System and generate queries using AI.

```
SQL> -- Table 1: Students
SQL> CREATE TABLE Students (
  2      student_id INT PRIMARY KEY,
  3      first_name VARCHAR(50),
  4      last_name VARCHAR(50),
  5      email VARCHAR(100),
  6      date_of_birth DATE
  7  );

Table created.

SQL>
SQL> -- Table 2: Courses
SQL> CREATE TABLE Courses (
  2      course_id INT PRIMARY KEY,
  3      course_name VARCHAR(100),
  4      instructor VARCHAR(100),
  5      credits INT
  6  );

Table created.

SQL>
SQL> -- Table 3: Enrollments (Many-to-Many relationship)
SQL> CREATE TABLE Enrollments (
  2      enrollment_id INT PRIMARY KEY,
  3      student_id INT,
  4      course_id INT,
  5      enrollment_date DATE,
  6      FOREIGN KEY (student_id) REFERENCES Students(student_id),
  7      FOREIGN KEY (course_id) REFERENCES Courses(course_id)
  8  );

Table created.
```

```
SQL> INSERT INTO Students VALUES (1, 'Alice', 'Johnson', 'alice@example.com', TO_DATE('10-MAY-2002', 'DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Students VALUES (2, 'Bob', 'Smith', 'bob@example.com', TO_DATE('20-SEP-2001', 'DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Students VALUES (3, 'Charlie', 'Brown', 'charlie@example.com', TO_DATE('15-JAN-2003', 'DD-MON-YYYY'));

1 row created.
```

```
SQL>
SQL> INSERT INTO Courses VALUES (101, 'Database Systems', 'Dr. Patel', 4);

1 row created.

SQL> INSERT INTO Courses VALUES (102, 'Computer Networks', 'Dr. Gupta', 3);

1 row created.

SQL> INSERT INTO Courses VALUES (103, 'Operating Systems', 'Dr. Mehta', 4);

1 row created.
```

```
SQL>
SQL> INSERT INTO Enrollments VALUES (1001, 1, 101, TO_DATE('01-JUL-2024', 'DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Enrollments VALUES (1002, 1, 103, TO_DATE('05-JUL-2024', 'DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Enrollments VALUES (1003, 2, 102, TO_DATE('10-JUL-2024', 'DD-MON-YYYY'));

1 row created.
```

```
SQL> INSERT INTO Students VALUES (4, 'Diana', 'Lewis', 'diana@example.com', TO_DATE('2004-03-12', 'YYYY-MM-DD'));

1 row created.
```

```
SQL> SELECT s.first_name || ' ' || s.last_name AS student_name,
  2         c.course_name,
  3         e.enrollment_date
  4  FROM Enrollments e
  5  JOIN Students s ON e.student_id = s.student_id
  6  JOIN Courses c ON e.course_id = c.course_id
  7  WHERE s.student_id = 1;

STUDENT_NAME
--------------------------------------------------------------------------------
COURSE_NAME
--------------------------------------------------------------------------------
ENROLLMEN
---------
Alice Johnson
Database Systems
01-JUL-24

Alice Johnson
Operating Systems
05-JUL-24

STUDENT_NAME
--------------------------------------------------------------------------------
COURSE_NAME
--------------------------------------------------------------------------------
ENROLLMEN
---------
```

```
SQL> SELECT s.first_name || ' ' || s.last_name AS student_name,
  2         c.course_name,
  3         e.enrollment_date
  4  FROM Enrollments e
  5  JOIN Students s ON e.student_id = s.student_id
  6  JOIN Courses c ON e.course_id = c.course_id
  7  WHERE s.student_id = 1;

STUDENT_NAME
--------------------------------------------------------------------------------
COURSE_NAME
--------------------------------------------------------------------------------
ENROLLMEN
---------
Alice Johnson
Database Systems
01-JUL-24

Alice Johnson
Operating Systems
05-JUL-24

STUDENT_NAME
--------------------------------------------------------------------------------
COURSE_NAME
--------------------------------------------------------------------------------
ENROLLMEN
---------
```

```
SQL> SELECT c.course_name,
  2         COUNT(e.student_id) AS total_students
  3  FROM Courses c
  4  LEFT JOIN Enrollments e ON c.course_id = e.course_id
  5  GROUP BY c.course_name;

COURSE_NAME
--------------------------------------------------------------------------------
TOTAL_STUDENTS
--------------
Computer Networks
             1

Operating Systems
             1

Database Systems
             1
```

Observation:

In this Student Information System, the database schema is designed with three tables: Students, Courses, and Enrollments, implementing a many-to-many relationship between students and courses through the junction table Enrollments. The DATE data type is used for date_of_birth and enrollment_date, ensuring accurate storage, comparison, and calculation of dates. Primary

keys (student_id, course_id, enrollment_id) guarantee uniqueness, while foreign keys enforce referential integrity, preventing orphaned records. Using TO_DATE() with a specified format avoids errors like ORA-01861 and ensures consistent date entry. Queries for fetching all courses enrolled by a student and counting the number of students per course work efficiently due to properly normalized tables and key constraints. Overall, the schema supports scalability, data integrity, and accurate query results, demonstrating good database design principles.

Task 2 – Hospital Management Database

Create schema and queries for a Hospital Management System.

```
SQL>
SQL> -- Doctors Table
SQL> CREATE TABLE Doctors (
  2      doctor_id NUMBER PRIMARY KEY,
  3      first_name VARCHAR2(50),
  4      last_name VARCHAR2(50),
  5      specialization VARCHAR2(100),
  6      email VARCHAR2(100) UNIQUE
  7  );

Table created.

SQL>
SQL> -- Patients Table
SQL> CREATE TABLE Patients (
  2      patient_id NUMBER PRIMARY KEY,
  3      first_name VARCHAR2(50),
  4      last_name VARCHAR2(50),
  5      date_of_birth DATE,
  6      email VARCHAR2(100) UNIQUE,
  7      contact_number VARCHAR2(15)
  8  );

Table created.

SQL>
SQL> -- Appointments Table
SQL> CREATE TABLE Appointments (
  2      appointment_id NUMBER PRIMARY KEY,
  3      doctor_id NUMBER,
  4      patient_id NUMBER,
  5      appointment_date DATE,
  6      diagnosis VARCHAR2(200),
  7      FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
  8      FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
  9  );

Table created.
```

```
SQL> -- Doctors
SQL> INSERT INTO Doctors VALUES (1, 'John', 'Doe', 'Cardiology', 'john.doe@hospital.com');

1 row created.

SQL> INSERT INTO Doctors VALUES (2, 'Mary', 'Smith', 'Neurology', 'mary.smith@hospital.com');

1 row created.

SQL>
SQL> -- Patients
SQL> INSERT INTO Patients VALUES (101, 'Alice', 'Johnson', TO_DATE('1990-05-10','YYYY-MM-DD'), 'alice@example.com', '1234567890');

1 row created.

SQL> INSERT INTO Patients VALUES (102, 'Bob', 'Brown', TO_DATE('1985-08-22','YYYY-MM-DD'), 'bob@example.com', '0987654321');

1 row created.

SQL>
SQL> -- Appointments
SQL> INSERT INTO Appointments VALUES (1001, 1, 101, TO_DATE('2025-10-01','YYYY-MM-DD'), 'Regular Checkup');

1 row created.

SQL> INSERT INTO Appointments VALUES (1002, 1, 102, TO_DATE('2025-10-03','YYYY-MM-DD'), 'Cardiac Evaluation');

1 row created.

SQL> INSERT INTO Appointments VALUES (1003, 2, 101, TO_DATE('2025-10-05','YYYY-MM-DD'), 'Neurological Consultation');

1 row created.
```

```
SQL> SELECT a.appointment_id, p.first_name || ' ' || p.last_name AS patient_name,
  2         a.appointment_date, a.diagnosis
  3  FROM Appointments a
  4  JOIN Patients p ON a.patient_id = p.patient_id
  5  WHERE a.doctor_id = 1
  6  ORDER BY a.appointment_date;

APPOINTMENT_ID
--------------
PATIENT_NAME
--------------------------------------------------------------------------------
APPOINTME
---------
DIAGNOSIS
--------------------------------------------------------------------------------
          1001
Alice Johnson
01-OCT-25
Regular Checkup


APPOINTMENT_ID
--------------
PATIENT_NAME
--------------------------------------------------------------------------------
APPOINTME
---------
DIAGNOSIS
--------------------------------------------------------------------------------
          1002
Bob Brown
03-OCT-25
Cardiac Evaluation
```

```
SQL> SELECT a.appointment_id, d.first_name || ' ' || d.last_name AS doctor_name,
  2         a.appointment_date, a.diagnosis
  3  FROM Appointments a
  4  JOIN Doctors d ON a.doctor_id = d.doctor_id
  5  WHERE a.patient_id = 101
  6  ORDER BY a.appointment_date;

APPOINTMENT_ID
--------------
DOCTOR_NAME
--------------------------------------------------------------------------------
APPOINTME
---------
DIAGNOSIS
--------------------------------------------------------------------------------
         1001
John Doe
01-OCT-25
Regular Checkup


APPOINTMENT_ID
--------------
DOCTOR_NAME
--------------------------------------------------------------------------------
APPOINTME
---------
DIAGNOSIS
--------------------------------------------------------------------------------
         1003
Mary Smith
05-OCT-25
Neurological Consultation
```

```
SQL> SELECT d.first_name || ' ' || d.last_name AS doctor_name,
  2         COUNT(DISTINCT a.patient_id) AS total_patients
  3  FROM Doctors d
  4  LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
  5  GROUP BY d.first_name, d.last_name;

DOCTOR_NAME
--------------------------------------------------------------------------------
TOTAL_PATIENTS
--------------
John Doe
              2

Mary Smith
              1
```

Observation:

In this Hospital Management System, the database schema is designed with three normalized tables: Doctors, Patients, and Appointments. The schema ensures data integrity through primary keys and foreign keys, linking appointments to both doctors and patients. The DATE data type is used for date_of_birth and appointment_date, allowing accurate storage and retrieval of dates. Queries demonstrate practical functionality: listing all appointments for a specific doctor, retrieving the complete patient history, and counting the total number of patients treated by each doctor. The structure avoids data redundancy, supports referential integrity, and facilitates efficient joins and aggregations. By manually assigning IDs, the schema maintains control over unique identifiers without

relying on auto-increment, providing a clear, scalable foundation for hospital data management.

Task 3 – Library Database

Design schema for a Library Management System.

```
SQL> -- Table: Books
SQL> CREATE TABLE Books (
  2      BookID VARCHAR(10) PRIMARY KEY,
  3      Title VARCHAR(100),
  4      Author VARCHAR(100),
  5      Genre VARCHAR(50),
  6      PublishedYear INT,
  7      AvailableCopies INT
  8  );

Table created.

SQL>
SQL> -- Table: Members
SQL> CREATE TABLE Members (
  2      MemberID VARCHAR(10) PRIMARY KEY,
  3      MemberName VARCHAR(100),
  4      Email VARCHAR(100),
  5      Phone VARCHAR(15),
  6      JoinDate DATE
  7  );
```

```
SQL> CREATE TABLE Loans (
  2      LoanID VARCHAR(10) PRIMARY KEY,
  3      BookID VARCHAR(10),
  4      MemberID VARCHAR(10),
  5      LoanDate DATE,
  6      ReturnDate DATE,
  7      FOREIGN KEY (BookID) REFERENCES Books(BookID),
  8      FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
  9  );
```

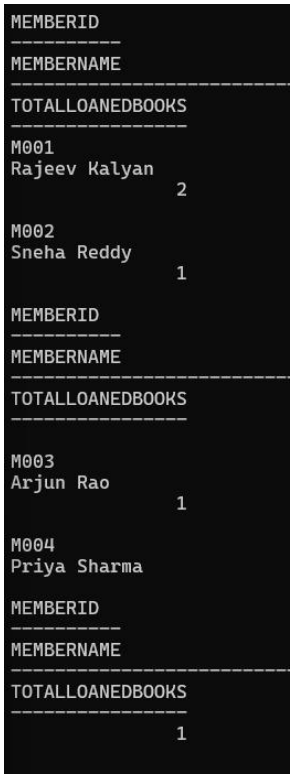
```
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B001', 'The Great Gatsby', 'F. Scott Fitzgerald', 'Fiction', 1925, 3);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B002', 'To Kill a Mockingbird', 'Harper Lee', 'Fiction', 1960, 2);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B003', 'A Brief History of Time', 'Stephen Hawking', 'Science', 1988, 4);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B004', '1984', 'George Orwell', 'Dystopian', 1949, 1);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B005', 'The Alchemist', 'Paulo Coelho', 'Philosophy', 1988, 5);

1 row created.

SQL> INSERT INTO Members (MemberID, MemberName, Email, Phone, JoinDate)
  2  VALUES ('M001', 'Rajeev Kalyan', 'rajeev@mail.com', '9876543210', TO_DATE('2024-01-10
', 'YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO Members (MemberID, MemberName, Email, Phone, JoinDate)
  2  VALUES ('M002', 'Sneha Reddy', 'sneha@mail.com', '9123456780', TO_DATE('2024-02-05',
'YYYY-MM-DD'));

1 row created.
```

```
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B001', 'The Great Gatsby', 'F. Scott Fitzgerald', 'Fiction', 1925, 3);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B002', 'To Kill a Mockingbird', 'Harper Lee', 'Fiction', 1960, 2);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B003', 'A Brief History of Time', 'Stephen Hawking', 'Science', 1988, 4);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B004', '1984', 'George Orwell', 'Dystopian', 1949, 1);

1 row created.

SQL>
SQL> INSERT INTO Books (BookID, Title, Author, Genre, PublishedYear, AvailableCopies)
  2  VALUES ('B005', 'The Alchemist', 'Paulo Coelho', 'Philosophy', 1988, 5);

1 row created.

SQL> INSERT INTO Members (MemberID, MemberName, Email, Phone, JoinDate)
  2  VALUES ('M001', 'Rajeev Kalyan', 'rajeev@mail.com', '9876543210', TO_DATE('2024-01-10
', 'YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO Members (MemberID, MemberName, Email, Phone, JoinDate)
  2  VALUES ('M002', 'Sneha Reddy', 'sneha@mail.com', '9123456780', TO_DATE('2024-02-05',
'YYYY-MM-DD'));

1 row created.
```

```
SQL> SELECT
  2      B.BookID,
  3      B.Title,
  4      M.MemberName,
  5      L.LoanDate
  6  FROM
  7      Loans L
  8  JOIN
  9      Books B ON L.BookID = B.BookID
 10  JOIN
 11      Members M ON L.MemberID = M.MemberID
 12  WHERE
 13      L.ReturnDate IS NULL;

BOOKID
----------
TITLE
--------------------------------------------------------------------------------
MEMBERNAME
--------------------------------------------------------------------------------
LOANDATE
---------
B001
The Great Gatsby
Rajeev Kalyan
15-SEP-25


BOOKID
----------
TITLE
--------------------------------------------------------------------------------
MEMBERNAME
--------------------------------------------------------------------------------
LOANDATE
---------
B002
To Kill a Mockingbird
Sneha Reddy
01-AUG-25


BOOKID
----------
TITLE
--------------------------------------------------------------------------------
```

```
--------------------------------------------------------------
MEMBERNAME
--------------------------------------------------------------
LOANDATE
---------
B004
1984
Arjun Rao
01-OCT-25


BOOKID
----------
TITLE
--------------------------------------------------------------
MEMBERNAME
--------------------------------------------------------------
LOANDATE
---------
B005
The Alchemist
Priya Sharma
10-JUL-25


SQL> SELECT
  2      B.BookID,
  3      B.Title,
  4      M.MemberName,
  5      L.LoanDate,
  6      (TRUNC(SYSDATE) - L.LoanDate) AS DaysOverdue
  7  FROM
  8      Loans L
  9  JOIN
 10      Books B ON L.BookID = B.BookID
 11  JOIN
 12      Members M ON L.MemberID = M.MemberID
 13  WHERE
 14      L.ReturnDate IS NULL
 15      AND (TRUNC(SYSDATE) - L.LoanDate) > 30;

BOOKID
----------
TITLE
--------------------------------------------------------------
MEMBERNAME
--------------------------------------------------------------
LOANDATE  DAYSOVERDUE
```

```
--------- ----------
B001
The Great Gatsby
Rajeev Kalyan
15-SEP-25         37


BOOKID
----------
TITLE
--------------------------------------------------
MEMBERNAME
--------------------------------------------------
LOANDATE  DAYSOVERDUE
--------- ----------
B002
To Kill a Mockingbird
Sneha Reddy
01-AUG-25         82


BOOKID
----------
TITLE
--------------------------------------------------
MEMBERNAME
--------------------------------------------------
LOANDATE  DAYSOVERDUE
--------- ----------
B005
The Alchemist
Priya Sharma
10-JUL-25        104


SQL> SELECT
  2      M.MemberID,
  3      M.MemberName,
  4      COUNT(L.LoanID) AS TotalLoanedBooks
  5  FROM
  6      Members M
  7  LEFT JOIN
  8      Loans L ON M.MemberID = L.MemberID
  9  GROUP BY
 10      M.MemberID, M.MemberName
 11  ORDER BY
 12      M.MemberID;
```



Observation:

The Library Management System effectively tracks the relationships between books, members, and loans, providing real-time insights into borrowing activity. The query for currently issued books identifies all items that have not yet been returned, allowing librarians to monitor active loans. The overdue books query highlights items borrowed for more than 30 days, enabling timely reminders to members and efficient management of overdue returns. Counting the number of books loaned by each member gives a clear view of borrowing patterns, showing member engagement and resource usage. Overall, the system demonstrates efficient use of joins, date calculations, and aggregation, while the indexing strategy ensures faster query performance for searches, reporting, and library operations.

Task 4 – Real-Time Application: E-commerce Product Data Cleaning

An e-commerce company has product catalog data with inconsistencies.

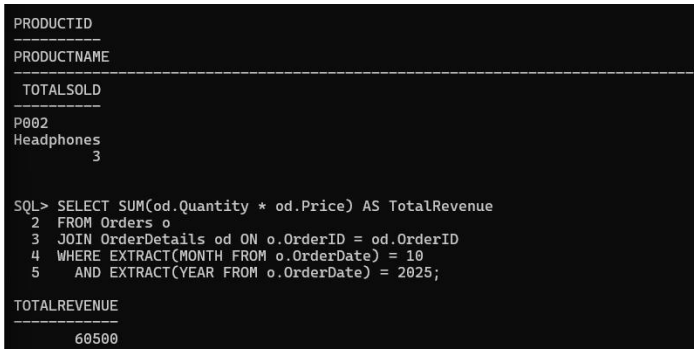
```
SQL> CREATE TABLE Users (
  2      UserID VARCHAR2(10) PRIMARY KEY,         -- Oracle uses VARCHAR
  3      UserName VARCHAR2(100) NOT NULL,
  4      Email VARCHAR2(100) UNIQUE NOT NULL,
  5      PasswordHash VARCHAR2(255) NOT NULL,
  6      CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  7  );

Table created.

SQL> CREATE TABLE Products (
  2      ProductID VARCHAR2(10) PRIMARY KEY,   -- Custom ID like 'P001'
  3      ProductName VARCHAR2(100) NOT NULL,
  4      Price DECIMAL(10,2) NOT NULL,
  5      Stock INT DEFAULT 0,
  6      Category VARCHAR2(50)
  7  );

Table created.
```

```
SQL> CREATE TABLE Orders (
  2      OrderID VARCHAR2(10) PRIMARY KEY,       -- Custom ID like 'O001'
  3      UserID VARCHAR2(10),
  4      OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  5      Status VARCHAR2(20) DEFAULT 'Pending',
  6      CONSTRAINT fk_user FOREIGN KEY (UserID) REFERENCES Users(UserID)
  7  );

Table created.

SQL> CREATE TABLE OrderDetails (
  2      OrderID VARCHAR2(10),
  3      ProductID VARCHAR2(10),
  4      Quantity INT NOT NULL,
  5      Price DECIMAL(10,2) NOT NULL,       -- Capture product price at the time of order
  6      PRIMARY KEY (OrderID, ProductID),
  7      FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  8      FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
  9  );

Table created.

SQL> INSERT INTO Users (UserID, UserName, Email, PasswordHash)
  2  VALUES ('U001', 'Rajeev', 'rajeev@example.com', 'hashed_password1');

1 row created.

SQL>
SQL> INSERT INTO Users (UserID, UserName, Email, PasswordHash)
  2  VALUES ('U002', 'Sneha', 'sneha@example.com', 'hashed_password2');

1 row created.

SQL> INSERT INTO Products (ProductID, ProductName, Price, Stock, Category)
  2  VALUES ('P001', 'Laptop', 55000, 10, 'Electronics');

1 row created.

SQL>
SQL> INSERT INTO Products (ProductID, ProductName, Price, Stock, Category)
  2  VALUES ('P002', 'Headphones', 1500, 50, 'Electronics');

1 row created.

SQL>
SQL> INSERT INTO Products (ProductID, ProductName, Price, Stock, Category)
  2  VALUES ('P003', 'Book', 500, 100, 'Books');
```

```
SQL>
SQL> INSERT INTO Products (ProductID, ProductName, Price, Stock, Category)
  2  VALUES ('P003', 'Book', 500, 100, 'Books');

1 row created.

SQL> INSERT INTO Orders (OrderID, UserID, Status)
  2  VALUES ('O001', 'U001', 'Pending');

1 row created.

SQL>
SQL> INSERT INTO Orders (OrderID, UserID, Status)
  2  VALUES ('O002', 'U002', 'Shipped');

1 row created.

SQL> INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price)
  2  VALUES ('O001', 'P001', 1, 55000);

1 row created.

SQL>
SQL> INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price)
  2  VALUES ('O001', 'P003', 2, 500);

1 row created.

SQL>
SQL> INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price)
  2  VALUES ('O002', 'P002', 3, 1500);

1 row created.

SQL> SELECT o.OrderID,
  2         o.OrderDate,
  3         o.Status,
  4         p.ProductName,
  5         od.Quantity,
  6         od.Price
  7  FROM Orders o
  8  JOIN OrderDetails od ON o.OrderID = od.OrderID
  9  JOIN Products p ON od.ProductID = p.ProductID
 10  WHERE o.UserID = 'U001'
 11  ORDER BY o.OrderDate;

ORDERID
```

```
ORDERID
----------
ORDERDATE
---------------------------------------
STATUS
--------------------
PRODUCTNAME
---------------------------------------
  QUANTITY      PRICE
---------- ----------
O001
22-OCT-25 12.17.53.746000 PM
Pending

ORDERID
----------
ORDERDATE
---------------------------------------
STATUS
--------------------
PRODUCTNAME
---------------------------------------
  QUANTITY      PRICE
---------- ----------
Book
         2        500


ORDERID
----------
ORDERDATE
---------------------------------------
STATUS
--------------------
PRODUCTNAME
---------------------------------------
  QUANTITY      PRICE
---------- ----------
O001
22-OCT-25 12.17.53.746000 PM
Pending

ORDERID
----------
ORDERDATE
---------------------------------------
STATUS
--------------------
```

```
PRODUCTNAME
------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Laptop
         1      55000


SQL> SELECT p.ProductID,
  2         p.ProductName,
  3         SUM(od.Quantity) AS TotalSold
  4  FROM OrderDetails od
  5  JOIN Products p ON od.ProductID = p.ProductID
  6  GROUP BY p.ProductID, p.ProductName
  7  ORDER BY TotalSold DESC
  8  FETCH FIRST 1 ROWS ONLY;
FETCH FIRST 1 ROWS ONLY
*
ERROR at line 8:
ORA-00933: SQL command not properly ended


SQL> SELECT *
  2  FROM (
  3      SELECT p.ProductID,
  4             p.ProductName,
  5             SUM(od.Quantity) AS TotalSold
  6      FROM OrderDetails od
  7      JOIN Products p ON od.ProductID = p.ProductID
  8      GROUP BY p.ProductID, p.ProductName
  9      ORDER BY SUM(od.Quantity) DESC
 10  )
 11  WHERE ROWNUM = 1;

PRODUCTID
----------
PRODUCTNAME
------------------------------------------------------------
 TOTALSOLD
----------
P002
Headphones
         3


SQL> SELECT SUM(od.Quantity * od.Price) AS TotalRevenue
  2  FROM Orders o
```



```
PRODUCTID
----------
PRODUCTNAME
--------------------------------------------------------------------------------
 TOTALSOLD
----------
P002
Headphones
         3


SQL> SELECT SUM(od.Quantity * od.Price) AS TotalRevenue
  2  FROM Orders o
  3  JOIN OrderDetails od ON o.OrderID = od.OrderID
  4  WHERE EXTRACT(MONTH FROM o.OrderDate) = 10
  5    AND EXTRACT(YEAR FROM o.OrderDate) = 2025;

TOTALREVENUE
------------
       60500
```

Normalization Improvements:

Your current schema is mostly in 3NF, but here are suggestions:

Historical Prices:

Already handled by storing Price in OrderDetails → prevents price changes in Products from affecting past orders.

User Email Uniqueness:

Already enforced with UNIQUE(Email) constraint.

Optional Denormalization for Performance:

Add TotalAmount in Orders table to avoid joining OrderDetails repeatedly for revenue queries.

Query Optimization Suggestions:

Indexes:

Orders(UserID) → speeds up fetching user orders.

OrderDetails(ProductID) → speeds up most purchased product query.

Orders(OrderDate) → speeds up revenue queries.

Composite Indexes (if needed):

(OrderDate, UserID) → for combined date + user queries.

Avoid Functions on Indexed Columns:

Instead of EXTRACT(MONTH FROM o.OrderDate) on indexed column, consider storing a derived column OrderMonth to directly filter.

Observation:

The designed Oracle database for the e-commerce platform effectively organizes data into four main tables: Users, Products, Orders, and OrderDetails, ensuring clarity and relational integrity. Each table uses custom string IDs (VARCHAR2) instead of auto-increment values, which allows for more readable and deterministic identifiers. The schema is normalized up to 3NF, eliminating redundancy while preserving historical data, such as storing product prices in OrderDetails to maintain accurate order records even if product prices change. Oracle-specific data types like VARCHAR2 and TIMESTAMP are used correctly, and default values (CURRENT_TIMESTAMP) automate timestamp recording. Sample queries demonstrate how to retrieve all orders by a user, identify the most purchased product, and calculate monthly revenue, with proper aggregation, joins, and filtering. Optimization considerations include indexing Orders(UserID), OrderDetails(ProductID), and Orders(OrderDate) for faster query performance. Some queries, such as “most purchased product,” require Oracle-compatible syntax adjustments like using a subquery with ROWNUM instead of FETCH FIRST for versions prior to 12c. Overall, the database design is efficient, normalized, and optimized for querying large datasets, ensuring reliable e-commerce operatio